PROGRAMMING

PROG6212 POE-PART ONE

STUDENT NAME : SUCCESS MALEFO
STUDENT NUMBER : ST10480235
ROSEBANK COLLEGE PTA

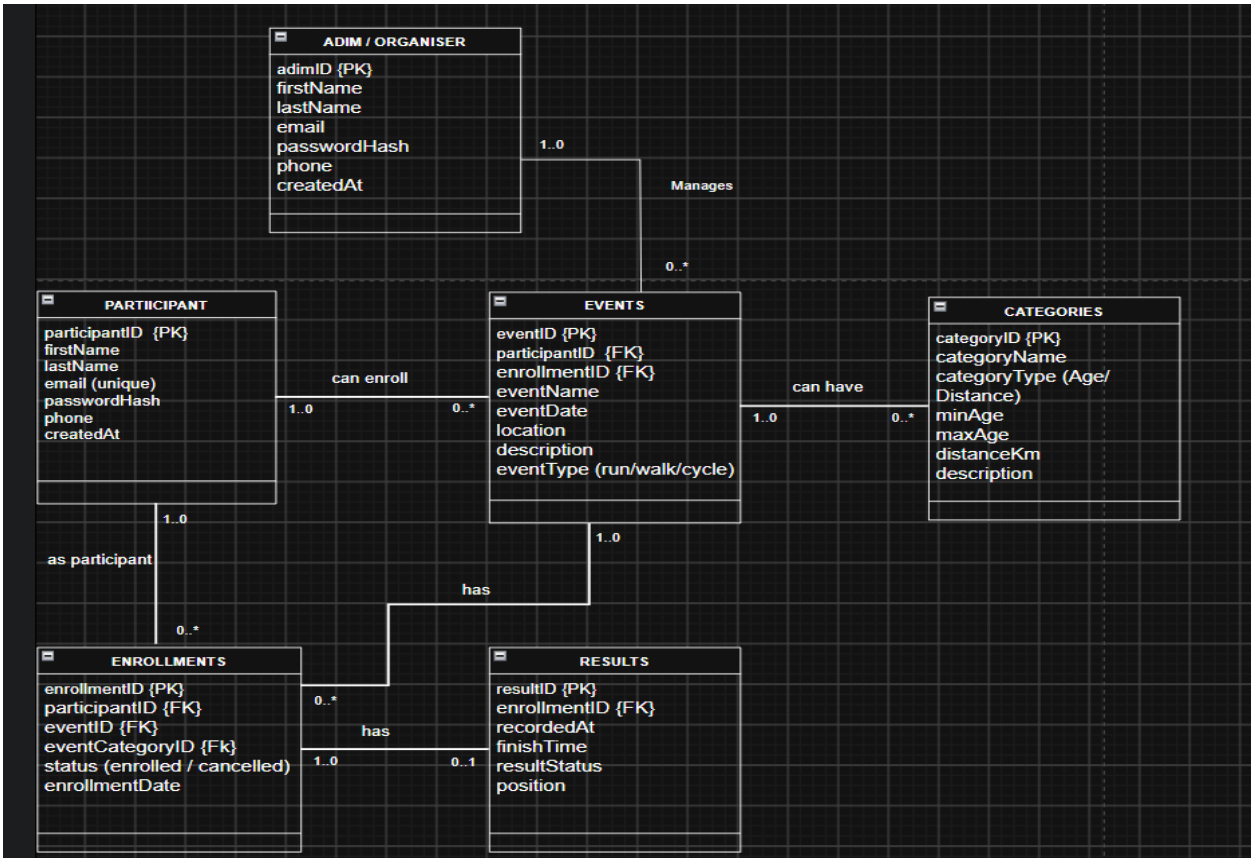
1. Introduction

The RaceDay system is being planned as a simple web-based system for organizing running, walking and cycling events. The main idea behind the design is to keep information organized so that organizers can manage events and results, while participants can create accounts, view events, choose categories and enroll. A well-designed database helps organize related information and maintain data consistency across the system (Connolly & Begg, 2015).

Section A – Entity Relationship Diagram

An Entity Relationship Diagram (ERD) is used to represent entities, their attributes and the relationships between entities in a database. The ERD helps to provide a clear view of how the different parts of the RaceDay database are connected (Coronel & Morris, 2019).

The ERD for the RaceDay system contains six entities. I decided to separate **Admin/Organizer** and **Participant** rather than keeping them together in one Users entity. This is because the two types of users perform different activities in the system. An Admin/Organizer is responsible for managing events and recording results, while a Participant mainly views events and enrolls in them.



Entity Descriptions

1. Admin/Organizer

The Admin/Organizer entity stores the details of people who are responsible for managing RaceDay events. An Admin/Organizer can create events, update event information and record participant results.

Main attributes:

- OrganizerID – INT, Primary Key
- FirstName – VARCHAR
- LastName – VARCHAR
- Email – VARCHAR, Unique
- Password – VARCHAR

2. Participant

The Participant entity stores information about people who take part in RaceDay events. Participants can view available events and enroll in events. Each participant can enroll in multiple events.

Main attributes:

- ParticipantID – INT, Primary Key
- FirstName – VARCHAR
- LastName – VARCHAR
- Email – VARCHAR, Unique
- Password – VARCHAR
- PhoneNumber – VARCHAR

3. Events

The Events entity stores information about the different running, walking and cycling events organized through the system. Each event is managed by an Admin/Organizer.

Main attributes:

- EventID – INT, Primary Key
- EventName – VARCHAR
- Description – VARCHAR
- EventDate – DATE
- Location – VARCHAR
- Distance – INT
- EventType – VARCHAR
- OrganizerID – INT, Foreign Key

4. Categories

The Categories entity stores the categories available for each event. Categories can be based on distance or participant group. For example, an event could have categories such as 5 km, 10 km and 21 km.

Main attributes:

- CategoryID – INT, Primary Key
- CategoryName – VARCHAR
- Description – VARCHAR
- EventID – INT, Foreign Key

5. Enrolments

Enrolments entity records when a participant enrolls in an event. It connects Participants with Events. This is necessary because one participant can enroll in many events, while one event can have many participants.

Main attributes:

- EnrolmentID – INT, Primary Key
- ParticipantID – INT, Foreign Key

- EventID – INT, Foreign Key
- EnrolmentDate – DATE
- Status – VARCHAR

6. Results

The Results entity stores the result achieved by a participant after completing an event. It records information such as finishing position and finishing time. A participant will only receive a result after the organizer has recorded it.

Main attributes:

- ResultID – INT, Primary Key
- EnrolmentID – INT, Foreign Key
- FinishTime – TIME
- Position – INT

Relationships and Cardinality

1. Admin/Organizer 1:M Events

One Admin/Organizer can manage many events, but each event is managed by one Admin/Organizer.

Cardinality: Admin/Organizer (1) ——— (M) Events

For example, one organizer could manage a 5 km race, a cycling event and a walking event.

2. Events 1:M Categories

One Event can have many categories, while each category belongs to one event.

Cardinality: Events (1) ——— (M) Categories

For example, one running event could have 5 km, 10 km and 21 km categories.

3. Participant 1:M Enrolments

One Participant can have many enrolments because they can participate in several different events.

Cardinality: Participant (1) ——— (M) Enrolments

For example, a participant could enroll in a running event this month and a cycling event the following month.

4. Events 1:M Enrolments

One Event can have many enrolments because many participants can register for the same event. Each enrolment belongs to one event.

Cardinality: Events (1) ——— (M) Enrolments

This relationship allows the system to keep a record of all participants who have enrolled in an event.

5. Enrolments 1:0..1 Results

Enrolment may have no result if the participant has not completed the event yet. Once the organizer records the participant's result, the enrolment can have one result.

Cardinality: Enrolments (1) ——— (0..1) Results

This prevents the system from requiring a result before the event has taken place.

SECTION B – RESTful API Endpoint Plan

The RaceDay API will provide different endpoints for participants and Admin/Organizers. The endpoints are divided into authentication, profiles, events, categories, enrolments and results. Access to certain endpoints will depend on the role of the person using the system.

1 Authentication Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new Participant or Admin/Organiser account.	Public	{ firstName, lastName, email, password, phone, role }	**201 Created** – account created. **400 Bad Request** – invalid information. **409 Conflict** – email already exists.
POST	/api/auth/login	Allows a registered Participant or Admin/Organizer to log into the RaceDay system.	Public	{ email, password, role }	**200 OK** – login successful. **401 Unauthorized** – incorrect login details.
POST	/api/auth/logout	Logs the currently logged-in user out of the system.	Any logged-in user	None	**204 No Content** – logout successful. **401 Unauthorized** – user is not logged in.

2. Participant Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/participants/profile	Displays the profile information of the currently logged-in user.	Participant / Organiser	None	**200 OK –** profile details returned**. 401 Unauthorized –** user is not logged in.
PUT	/api/participants/profile	Allows a participant to update their own profile.	Participant	firstName, lastName, email, phone	**200 OK –** profile updated**. 400 Bad Request** – invalid data.
POST	/api/events/{eventId}/enrolments	Allows a participant to enrol in an event and select a category.	Participant	categoryId	**201 Created –** enrolment successful. **400 Bad Request –** invalid category. **409 Conflict –** already enrolled.
GET	/api/participants/enrolments	Displays the events in which the logged-in participant is enrolled.	Participant	None	**200 OK –** enrolments returned.
DELETE	/api/enrolments/{id}	Allows a participant to cancel their own enrolment.	Participant	None	**204 No Content –** enrolment cancelled. **403 Forbidden –** not their enrolment. **404 Not Found –** enrolment not found.

GET	/api/participants/results	Allows a participant to view their own race results.	Participant	None	**200 OK –** results returned.

3. Admin/Organizer Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/organizers/profile	Allows an Admin/Organizer to view their own profile	**Admin/Organizer**	None	**200 OK** – profile returned.
PUT	/api/organizers/profile	Allows an Admin/Organizer to update their own profile.	Admin/Organizer	**firstName ,lastName ,email,ph one**	**200 OK** – profile updated. **400 Bad Request** – invalid data.
POST	/api/events	Creates a new raceDay event.	Admin/Organizer	eventNam e,descripti on,eventD ate,locatio n,distance ,eventTyp e	**201 Created** – event created. **400 Bad Request** – invalid data. **403 Forbidden** – not authorized.
PUT	/api/events/{id}	Updates an event managed by the Admin/Organizer	Admin/Organizer	eventNam e,descripti on,eventD ate,locatio n,distance ,eventTyp e	**200 OK** – event updated. **403 Forbidden** – event does not belong to organizer. **404 Not Found** – event not found
DELETE	/api/events/{id}	Deletes an event managed by the Admin/organizer.	Admin/Organizer	None	**204 No Content** – enrolment cancelled. **403 Forbidden** – enrolment belongs to another user. **404 Not Found** – enrolment does not exist.

GET	/api/events/{eventId}/ enrolments	Allows an Organizer to view Participants enrolled in their event.	Admin/Organizer	None	**200 OK** – enrolment list returned. **403 Forbidden** – event is not managed by the Organiser.

4. Event Endpoints

HTTP Method	**Route**	**Description**	**Role Required**	**Request Body**	**Expected Response**
GET	/api/events	Displays all available RaceDay events.	Participant / Organiser	None	200 OK – list of events returned.
GET	/api/events/{id}	Displays the details of one specific event.	Participant / Organiser	None	200 OK – event details returned. 404 Not Found – event does not exist.
POST	/api/events	Creates a new RaceDay event.	Organiser	{ eventName, description, eventDate, location, distanceKm, eventType }	201 Created – event created. 400 Bad Request – invalid information. 403 Forbidden – user is not an Organiser.
PUT	/api/events/{id}	Updates an event managed by the Organiser.	Organiser	{ eventName, description, eventDate, location, distanceKm, eventType }	200 OK – event updated. 403 Forbidden – not authorised. 404 Not Found – event does not exist.
DELETE	/api/events/{id}	Removes an event managed by the Organiser.	Organiser	None	204 No Content – event deleted. 403 Forbidden – not authorised. 404 Not Found – event does not exist.

5. Category Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Displays all categories available for a specific event.	Participant / Admin/Organizer	None	200 OK – categories returned. 404 Not Found – event does not exist.
POST	/api/events/{eventId}/categories	Adds a new category to an event managed by the Organizer.	Admin/Organizer	{ categoryName, categoryType, minAge, maxAge, distanceKm, description }	201 Created – category created. 400 Bad Request – invalid information. 403 Forbidden – not authorised.
PUT	/api/categories/{id}	Updates an existing event category belonging to the organizer's event.	Admin/Organizer	{ categoryName, categoryType, minAge, maxAge, distanceKm, description }	200 OK – category updated. 403 Forbidden – not authorised. 404 Not Found – category does not exist.
DELETE	/api/categories/{id}	Removes a category from an event.	Admin/Organiser	None	204 No Content – category deleted. 403 Forbidden – not authorised. 404 Not Found –

					category does not exist.

6. Results Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/result	Records the result of a Participant who completed an event.	Admin/Organizer	{ finishTime, position, resultStatus }	201 Created – result recorded. 404 Not Found – enrolment does not exist. 409 Conflict – result already exists.
PUT	/api/enrolments/{enrolmentId}/result	Allows an Organiser to correct or update an existing result.	Admin/Organizer	{ finishTime, position, resultStatus }	200 OK – result updated. 403 Forbidden – not authorised. 404 Not Found – result does not exist.
GET	/api/participants/results	Allows a Participant to view their own race results.	Participant	None	200 OK – results returned. 401 Unauthorized – user is not logged in.
GET	/api/events/{eventId}/results	Displays the results for an event managed by the Organiser.	Admin/Organizer	None	200 OK – results returned. 403 Forbidden – not authorised. 404 Not Found – event does not exist.

7. Endpoint Planning Notes

The endpoint design follows the responsibilities of the two roles in the RaceDay system. **Participants** are mainly responsible for managing their own accounts, viewing events, enrolling in events and viewing their results. **Admin/Organizers** are responsible for creating and managing events, managing categories and recording results.

Authentication endpoints are public because a person needs to register and log in before accessing protected parts of the system. Once logged in, the API will use role-based access to determine which operations the person is allowed to perform.

For security, participants will only be allowed to view or update **their own information and enrolments**. They will not be allowed to create, update or delete events, categories or results. Similarly, an Admin/Organizer will only be allowed to edit or delete events that they manage.

The API will also check that a participant is enrolling in a valid category belonging to the selected event. This helps prevent incorrect relationships from being stored in the database.

For results, an Admin/Organizer can only record or update results for participants enrolled in an event that they manage. Participants can view their own results but cannot change them.

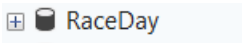
The API will return suitable HTTP status codes, such as **200 OK**, **201 Created**, **204 No Content**, **400 Bad Request**, **401 Unauthorized**, **403 Forbidden** and **404 Not Found** (MDN Web Docs, 2026). These responses will help the MVC front end in Part 3 understand whether a request was successful or why it could not be completed.

Section C – SQL Database Script

The SQL script supplied with this submission creates the RaceDayDB database and the six tables used in the ERD. It also adds primary keys, foreign keys, unique constraints and basic CHECK constraints. The sample passwords are placeholders because real passwords should be hashed by the application rather than stored in plain text.

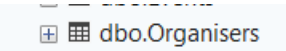
The full script is supplied separately as: RaceDay_Database.sql

```
CREATE DATABASE RaceDay;
```

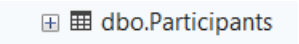



```
CREATE TABLE Organisers (
    OrganiserID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    FirstName VARCHAR(70) NOT NULL,
    LastName VARCHAR(60) NOT NULL,
    Email VARCHAR(200) NOT NULL UNIQUE,
    PasswordHash VARCHAR(295) NOT NULL,
    Phone VARCHAR(40) NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()
);
```

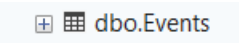



```
CREATE TABLE Participants (
    ParticipantID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(90) NOT NULL,
    LastName VARCHAR(60) NOT NULL,
    Email VARCHAR(200) NOT NULL UNIQUE,
    PasswordHash VARCHAR(295) NOT NULL,
    Phone VARCHAR(50) NULL,
    DateOfBirth DATE NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE()
);
```

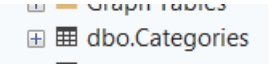



```
CREATE TABLE Events (
    EventID INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserID INT NOT NULL,
    EventName VARCHAR(150) NOT NULL,
    Description VARCHAR(275) NULL,
    EventDate DATETIME NOT NULL,
```

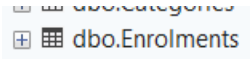
```
    Location VARCHAR(140) NOT NULL,
    DistanceKm DECIMAL(5,2) NOT NULL,
    EventType VARCHAR(90) NOT NULL,
    CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Organisers FOREIGN KEY (OrganiserID)
        REFERENCES Organisers(OrganiserID),
    CONSTRAINT CK_Events_Distance CHECK (DistanceKm > 0),
    CONSTRAINT CK_Events_Type CHECK (EventType IN ('Run','Walk','Cycle'))
);
```




```
CREATE TABLE Categories (
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,
    EventID INT NOT NULL,
    CategoryName VARCHAR(300) NOT NULL,
    CategoryType VARCHAR(60) NOT NULL,
    MinAge INT NULL,
    MaxAge INT NULL,
    DistanceKm DECIMAL(5,2) NULL,
    Description VARCHAR(355) NULL,
    CONSTRAINT FK_Categories_Events FOREIGN KEY (EventID)
        REFERENCES Events(EventID) ON DELETE CASCADE,
    CONSTRAINT CK_Categories_Type CHECK (CategoryType IN ('Age','Distance')),
    CONSTRAINT UQ_Event_CategoryName UNIQUE (EventID, CategoryName)
);
```



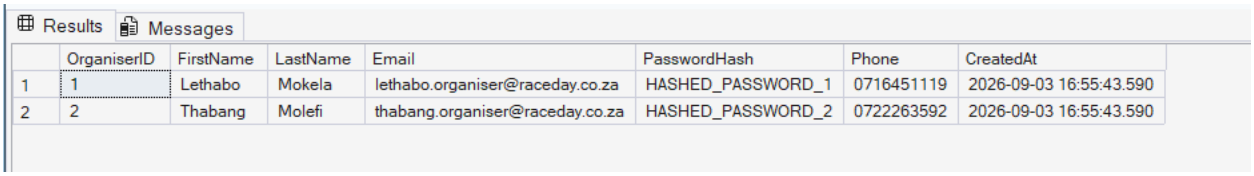
```
CREATE TABLE Enrolments (
    EnrolmentID INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantID INT NOT NULL,
    EventID INT NOT NULL,
    CategoryID INT NOT NULL,
    EnrolmentDate DATETIME NOT NULL DEFAULT GETDATE(),
    Status VARCHAR(70) NOT NULL DEFAULT 'Enrolled',
    CONSTRAINT FK_Enrolments_Participants FOREIGN KEY (ParticipantID)
        REFERENCES Participants(ParticipantID),
    CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventID)
        REFERENCES Events(EventID),
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryID)
        REFERENCES Categories(CategoryID),
    CONSTRAINT CK_Enrolments_Status CHECK (Status IN ('Enrolled','Cancelled')),
    CONSTRAINT UQ_Participant_Event UNIQUE (ParticipantID, EventID)
);
```




```
CREATE TABLE Results (
    ResultID INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentID INT NOT NULL UNIQUE,
    FinishTime TIME NULL,
    Position INT NULL,
    ResultStatus VARCHAR(80) NOT NULL DEFAULT 'Completed',
    RecordedAt DATETIME NOT NULL DEFAULT GETDATE(),
```

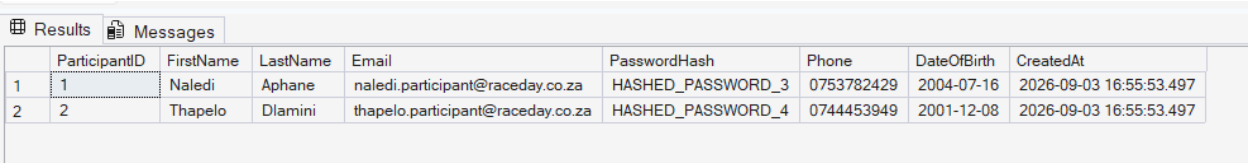
```
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentID)
        REFERENCES Enrolments(EnrolmentID) ON DELETE CASCADE,
    CONSTRAINT CK_Results_Position CHECK (Position IS NULL OR Position > 0),
    CONSTRAINT CK_Results_Status CHECK
        (ResultStatus IN ('Completed','Did Not Finish','Disqualified'))
);
```


```
INSERT INTO Organisers (FirstName, LastName, Email, PasswordHash, Phone)
VALUES
('Lethabo','Mokela','lethabo.organiser@raceday.co.za','HASHED_PASSWORD_1','0716451119'),
('Thabang','Molefi','thabang.organiser@raceday.co.za','HASHED_PASSWORD_2','0722263592');
```



	OrganiserID	FirstName	LastName	Email	PasswordHash	Phone	CreatedAt
1	1	Lethabo	Mokela	lethabo.organiser@raceday.co.za	HASHED_PASSWORD_1	0716451119	2026-09-03 16:55:43.590
2	2	Thabang	Molefi	thabang.organiser@raceday.co.za	HASHED_PASSWORD_2	0722263592	2026-09-03 16:55:43.590

```
INSERT INTO Participants (FirstName, LastName, Email, PasswordHash, Phone, DateOfBirth)
VALUES
('Naledi','Aphane','naledi.participant@raceday.co.za','HASHED_PASSWORD_3','0753782429','2004-07-16'),
('Thapelo','Dlamini','thapelo.participant@raceday.co.za','HASHED_PASSWORD_4','0744453949','2001-12-08');
```



	ParticipantID	FirstName	LastName	Email	PasswordHash	Phone	DateOfBirth	CreatedAt
1	1	Naledi	Aphane	naledi.participant@raceday.co.za	HASHED_PASSWORD_3	0753782429	2004-07-16	2026-09-03 16:55:53.497
2	2	Thapelo	Dlamini	thapelo.participant@raceday.co.za	HASHED_PASSWORD_4	0744453949	2001-12-08	2026-09-03 16:55:53.497

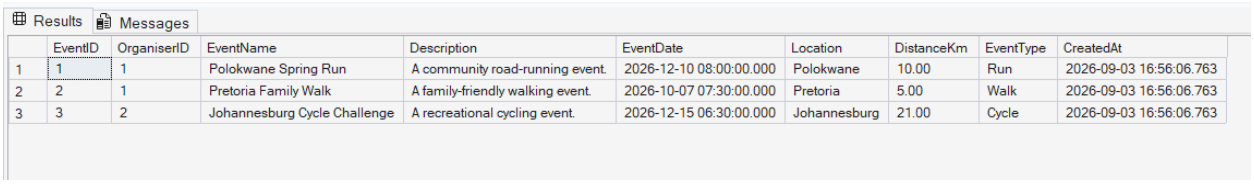
INSERT INTO Events (OrganiserID, EventName, Description, EventDate, Location, DistanceKm, EventType)

VALUES

(1,'Polokwane Spring Run','A community road-running event.','2026-12-10 08:00:00','Polokwane',10.00,'Run'),

(1,'Pretoria Family Walk','A family-friendly walking event.','2026-10-07 07:30:00','Pretoria',5.00,'Walk'),

(2,'Johannesburg Cycle Challenge','A recreational cycling event.','2026-12-15 06:30:00','Johannesburg',21.00,'Cycle');



Results Messages

	EventID	OrganiserID	EventName	Description	EventDate	Location	DistanceKm	EventType	CreatedAt
1	1	1	Polokwane Spring Run	A community road-running event.	2026-12-10 08:00:00.000	Polokwane	10.00	Run	2026-09-03 16:56:06.763
2	2	1	Pretoria Family Walk	A family-friendly walking event.	2026-10-07 07:30:00.000	Pretoria	5.00	Walk	2026-09-03 16:56:06.763
3	3	2	Johannesburg Cycle Challenge	A recreational cycling event.	2026-12-15 06:30:00.000	Johannesburg	21.00	Cycle	2026-09-03 16:56:06.763

INSERT INTO Categories (EventID, CategoryName, CategoryType, MinAge, MaxAge, DistanceKm, Description)

VALUES

(1,'Under 18','Age',0,19,NULL,'Participants younger than 18 years'),

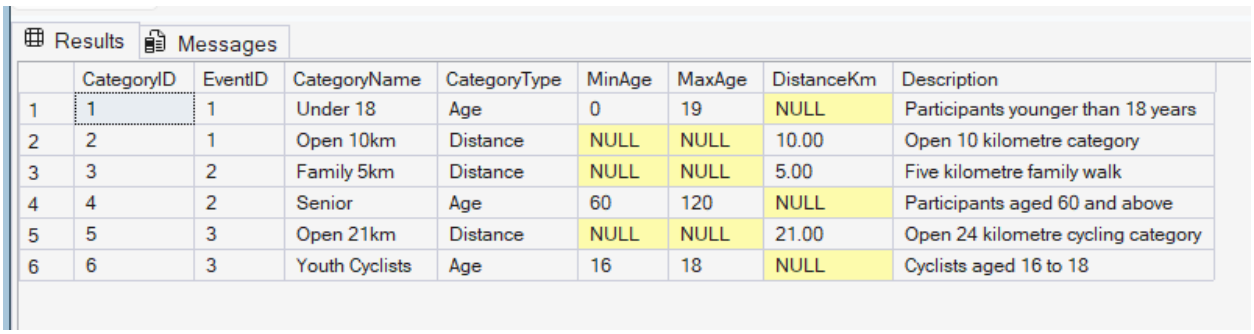
(1,'Open 10km','Distance',NULL,NULL,10.00,'Open 10 kilometre category'),

(2,'Family 5km','Distance',NULL,NULL,5.00,'Five kilometre family walk'),

(2,'Senior','Age',60,120,NULL,'Participants aged 60 and above'),

(3,'Open 21km','Distance',NULL,NULL,21.00,'Open 24 kilometre cycling category'),

(3,'Youth Cyclists','Age',16,18,NULL,'Cyclists aged 16 to 18');



Results Messages

	CategoryID	EventID	CategoryName	CategoryType	MinAge	MaxAge	DistanceKm	Description
1	1	1	Under 18	Age	0	19	NULL	Participants younger than 18 years
2	2	1	Open 10km	Distance	NULL	NULL	10.00	Open 10 kilometre category
3	3	2	Family 5km	Distance	NULL	NULL	5.00	Five kilometre family walk
4	4	2	Senior	Age	60	120	NULL	Participants aged 60 and above
5	5	3	Open 21km	Distance	NULL	NULL	21.00	Open 24 kilometre cycling category
6	6	3	Youth Cyclists	Age	16	18	NULL	Cyclists aged 16 to 18

```
INSERT INTO Enrolments (ParticipantID, EventID, CategoryID, Status)
VALUES
(1,1,1,'Enrolled'),
(2,1,2,'Enrolled'),
(1,2,3,'Enrolled'),
(2,3,5,'Enrolled');
```

Results | Messages

	EnrolmentID	ParticipantID	EventID	CategoryID	EnrolmentDate	Status
1	1	1	1	1	2026-09-03 16:56:49.267	Enrolled
2	2	2	1	2	2026-09-03 16:56:49.267	Enrolled
3	3	1	2	3	2026-09-03 16:56:49.267	Enrolled
4	4	2	3	5	2026-09-03 16:56:49.267	Enrolled

```
INSERT INTO Results (EnrolmentID, FinishTime, Position, ResultStatus)
VALUES
(1,'00:54:32',4,'Completed'),
(2,'01:02:15',9,'Completed');
```

Results | Messages

	ResultID	EnrolmentID	FinishTime	Position	ResultStatus	RecordedAt
1	1	1	00:54:32.0000000	4	Completed	2026-09-03 16:56:57.340
2	2	2	01:02:15.0000000	9	Completed	2026-09-03 16:56:57.340

4. Security and data-design considerations

The design treats the user's role as part of the account information because role-based access is a requirement of the RaceDay system. The API will use the authenticated session to determine the user's identity and role. Passwords will not be stored in their original form. Instead, Part 2 will store a password hash and verify the supplied password during login.

The database also uses foreign keys to prevent records such as enrolments and results from pointing to users, events or categories that do not exist. The unique constraint on (ParticipantId, EventCategoryId) is intended to stop the same participant from enrolling twice in the same event category.

5. Conclusion

The Part 1 plan gives me a clear structure to follow when developing the API. The ERD defines the data and relationships, while the endpoint table defines how the application will access that data. Keeping these two parts aligned should make the Part 2 implementation easier to test and should also reduce changes when the MVC front end is developed in Part 3.

References

Connolly, T. & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management*. 6th ed. Pearson.

Coronel, C. & Morris, S. (2019). *Database systems: Design, implementation, & management*. 13th ed. Cengage Learning.

Fielding, R.T. (2000). *Architectural styles and the design of network-based software architectures*. Doctoral dissertation, University of California, Irvine.

Microsoft. (2026). *Primary and foreign key constraints*. Microsoft Learn.

Microsoft. (2026). *Create primary keys and foreign keys*. Microsoft Learn.

MDN Web Docs. (2026). *HTTP response status codes*. Mozilla.

OWASP. (2026). *Password storage cheat sheet*. Open Worldwide Application Security Project.

OWASP. (2026). *Authorization cheat sheet*. Open Worldwide Application Security Project.